综合论文训练

AIoT 操作系统性能优化关键技术研究

系　　别：计算机科学与技术系

专　　业：计算机科学与技术

姓　　名：王　　铮

指导教师：陈　　渝　副教授

二〇二六年三月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定，即：学校有权保留论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

作者签名：　　导师签名：

日　　期：　　日　　期：

摘　要

【待完善】填写中文摘要

关键词：AIoT；操作系统；性能优化；ArceOS；Starry

Abstract

【待完善】 填写英文摘要

Keywords: AIoT; Operating System; Performance Optimization; ArceOS; Starry

目 录

第 1 章 绪论 1
1.1 研究背景 1
1.1.1 操作系统内核架构的演进 1
1.1.2 组件化操作系统与 ArceOS 1
1.1.3 AIoT 场景需求 2
1.1.4 Starry OS 的发展 2
1.2 问题陈述 2
1.3 相关工作 3
1.4 本文工作 4
1.5 论文结构安排 4
第 2 章 系统背景与总体架构 5
2.1 设计目标 5
2.2 ArceOS 基础与 StarryOS 定位 5
2.2.1 ArceOS 的灵活内核理念 5
2.2.2 ArceOS 的组件层次结构 6
2.2.3 StarryOS 的系统定位 6
2.3 系统总体架构 7
2.3.1 总体分层与模块关系 7
2.3.2 组件边界与仓库组织 7
2.4 内核执行模型 7
2.4.1 用户态与内核态的基本切换流程 7
2.4.2 进程、线程与任务对象关系 8
2.5 核心资源管理对象 9
2.5.1 地址空间与内存对象 9
2.5.2 文件、设备与 I/O 对象 9
2.5.3 通信与事件对象 10
2.6 本章小结 10
第 3 章 核心子系统设计与实现 11
3.1 系统调用框架 11
3.2 任务与进程管理 11

3.3 内存管理 13
3.4 文件系统 14
3.5 网络与终端子系统 14
3.6 信号与同步机制 15
3.7 本章小结 16
第 4 章 关键机制与性能优化 18
4.1 Rust 组件化开发的挑战 18
4.2 extern_trait 机制 18
4.2.1 设计动机 18
4.2.2 实现原理 19
4.2.3 应用场景 22
4.3 用户态切换与反转控制流 23
4.3.1 传统宏内核控制流的局限 23
4.3.2 反转控制流设计 24
4.3.3 基于 axcpu 的实现 25
4.4 高效内核-用户交互机制 26
4.4.1 用户内存访问机制 26
4.4.2 缓冲区抽象设计 26
4.5 异步 I/O 机制 26
4.5.1 设计目标 26
4.5.2 同步与异步混合架构 27
4.5.3 与信号和硬件的集成 27
4.6 多级缓存优化 27
4.7 多核调度与虚拟内存优化方向 27
4.7.1 多核调度优化 27
4.7.2 虚拟内存优化 27
4.8 本章小结 27
第 5 章 AI 应用支持探索 28
5.1 目标平台与硬件环境 28
5.2 NPU 驱动设计 28
5.3 AI 应用运行支持 28
5.4 当前进展与后续计划 28
第 6 章 实验评测 29
6.1 实验环境 29

6.2 功能完整性评测 29
6.2.1 系统调用测例 29
6.2.2 应用兼容性测试 29
6.3 性能评测 29
6.3.1 系统调用开销 30
6.3.2 I/O 性能 30
6.4 优化效果分析 30
6.5 开发工作量统计 30
6.6 本章小结 30
第 7 章 总结与展望 31
7.1 工作总结 31
7.2 未来工作展望 31
参考文献 32
附录 A 外文资料的调研阅读报告（或书面翻译） 34
附录 B 其他附录内容 35
致 谢 36
声 明 37

插图清单

附表清单

符号和缩略语说明

AIoT	Artificial Intelligence of Things，人工智能物联网
OS	Operating System，操作系统

第 1 章 绪论

1.1 研究背景

1.1.1 操作系统内核架构的演进

宏内核、微内核和Unikernel是操作系统研究中最具代表性的三类内核架构。宏内核将进程管理、内存管理、文件系统、网络协议栈和设备驱动等大多数核心功能置于同一特权地址空间内实现，其优势在于模块间调用路径短、运行时开销低，因而长期在通用计算场景中占据主导地位；但其缺点也较为明显，即系统规模扩大后内部耦合增强，故障隔离与安全边界相对薄弱。微内核则将尽可能少的机制保留在内核态，仅在内核中维护地址空间、线程调度和进程间通信等最基本功能，而将文件系统、驱动等服务下沉到用户态。该设计有利于提高可靠性、可维护性和隔离性，但频繁的跨边界通信会带来额外开销，实现复杂度也更高。

Unikernel 则进一步面向特定应用进行裁剪，将应用与所需的最小操作系统服务整体编译为单一镜像运行。与通用操作系统相比，Unikernel能够显著减少系统体积、缩短启动时间，并降低运行路径中的冗余功能开销，因而在云原生、边缘设备和专用化部署场景中具有吸引力。然而，这种架构通常以牺牲通用性和兼容性为代价，应用迁移与调试支持也相对不足。总体来看，三类架构分别在性能、隔离性、灵活性和兼容性之间做出不同权衡，并不存在适用于全部场景的统一最优方案。随着 AIoT、云边协同等场景对系统形态提出多样化需求，如何在不同架构之间复用基础能力、降低定制成本，成为操作系统设计中的一个重要问题。

1.1.2 组件化操作系统与 ArceOS

组件化操作系统的核心思想，是将内核中的基础服务按照清晰的接口边界拆分为可组合、可替换、可复用的独立组件，从而降低系统耦合度并提升定制效率。与传统以单一内核形态为中心进行扩展的方式相比，组件化设计更强调将调度、内存、文件系统、设备抽象等共性能力沉淀为稳定模块，再根据具体场景组织成不同系统形态。对于需要在性能、隔离性、兼容性和部署成本之间反复权衡的现代应用场景而言，这种设计方法具有较高的工程价值。

ArceOS 正是在这一背景下提出的实验内核。其基本出发点并非围绕某一种固定架构优化，而是将不同内核形态共享的底层服务视为“内核意义上的库”，通过统一的组件边界和初始化后执行逻辑的差异，构造出 Unikernel、宏内核乃

至 Hypervisor 等不同系统形态。这一设计理念可概括为 “One Architecture to build all”，即通过一套统一的组件化基础设施支撑多种内核形态的快速构建。该思路为后续在 ArceOS 基础上扩展兼容性内核提供了方法论基础，也为在不同场景间复用已有内核能力提供了新的实现路径。

1.1.3 AIoT 场景需求

AIoT 场景兼具通用计算与嵌入式计算的双重特征。一方面，边缘侧设备往往需要运行日益复杂的应用框架、网络协议和数据处理逻辑，这要求操作系统具备较好的接口兼容性与应用承载能力；另一方面，设备端的算力、内存和能耗预算通常受到严格限制，系统又必须尽可能降低运行时开销，并对 I/O、调度和内存访问路径进行精细优化。因此，面向 AIoT 场景的操作系统不能仅追求功能完整，也不能仅满足轻量化部署，而需要在兼容性、性能和资源利用率之间取得平衡。

与此同时，AIoT 部署环境通常呈现出硬件平台多样、应用需求变化快以及软硬件协同紧密等特点。操作系统不仅需要支持跨架构移植和设备适配，还应具备较强的模块化与可扩展能力，以便随着应用场景变化快速裁剪、替换或新增内核功能。尤其当系统需要进一步支持 NPU、DMA、高速通信接口等专用硬件能力时，内核内部的接口边界是否清晰、关键子系统是否便于复用，都会直接影响后续演进成本。基于这些考虑，AIoT 场景中的操作系统设计天然需要兼顾组件化、兼容性与性能优化三方面目标。

1.1.4 Starry OS 的发展

【待完善】 梳理 Starry-Old → Starry-Next → Starry Mix → StarryOS 主线的演进历程

1.2 问题陈述

综合上述背景与需求分析，面向 AIoT 场景的操作系统设计面临以下核心挑战。

首先，现有操作系统方案难以同时满足兼容性、轻量化与组件化三方面要求。传统 Linux 等通用操作系统虽然具备完整的 POSIX 兼容性和丰富的软件生态支持，但其宏内核架构带来的系统体积、启动时间和运行时开销对资源受限的边缘设备形成显著压力。另一方面，Unikernel 等专用化内核虽然能够实现极高的部署效率和运行性能，但其单一应用绑定模式和薄弱的兼容性支持使得现有软件迁移成本

高昂，难以适配 AIoT 场景中多样化的应用需求。微内核架构虽然提供了良好的隔离性与可扩展性，但其跨边界通信开销和实现复杂度同样不容忽视。

其次，组件化内核在构建完整兼容性内核时面临一系列技术难题。ArceOS 提供了灵活的组件化基础，但如何在此基础上构建支持进程管理、虚拟内存、文件系统、网络协议栈、信号机制等完整 POSIX 语义的宏内核，涉及系统调用路径设计、用户态与内核态切换、资源对象抽象、事件与同步机制等一系列工程挑战。现有组件化设计主要面向 Unikernel 等单一应用形态，对多进程、多线程、地址空间隔离等传统宏内核核心能力的支持相对薄弱。

再次，兼容性内核在关键路径上存在显著性能瓶颈。系统调用开销、用户态内存访问、I/O 处理延迟、缓存效率等问题直接关系到系统整体性能表现。如何在保持组件化架构清晰边界的前提下，对系统调用路径、内核-用户数据交互、异步 I/O 处理、多级缓存等关键机制进行优化，是构建高性能兼容性内核必须解决的问题。

最后，面向 AIoT 场景的专用硬件支持与 AI 应用运行时集成尚处于探索阶段。NPU、DMA 等专用加速器的驱动集成，以及 AI 推理框架在定制化内核上的运行支持，都需要在操作系统层面提供相应的抽象与接口。

针对上述问题，本文的研究工作围绕以下核心目标展开：在 ArceOS 组件化基座之上，构建一个具备 Linux 应用兼容性的宏内核系统，同时在关键路径上实现性能优化，并探索面向 AIoT 场景的专用能力扩展路径。

1.3 相关工作

在 Rust 操作系统方面，Asterinas 提出 framekernel 架构，通过内核内部特权分离实现最小的可信计算基，以安全 Rust 实现完整的 Linux ABI 兼容性[1]。Theseus 探索 intralingual 设计原则，将操作系统职责通过语言级机制委托给编译器，实现细粒度组件化和运行时动态演化[2]。Tock 面向嵌入式微控制器场景，通过 Rust 的类型安全实现内核与应用的隔离，支持在资源受限设备上安全运行多个互不信任的应用[3]，并已在超过一千万台设备上部署应用[4]。

在 Linux 兼容性内核方面，gVisor 采用用户态内核（application kernel）架构，以 Go 语言实现 Linux 系统调用接口，为容器提供强隔离沙箱环境[5]。Graphene（现名 Gramine）和 Occlum 均面向可信执行环境（TEE）场景，通过库操作系统（library OS）方式在 Intel SGX enclave 内运行多进程 Linux 应用[6]，后者以 Rust 实现内存安全[7]。

在性能优化方面，CortenMM 针对虚拟内存管理的正确性开销问题，提出取消独立 VMA 抽象的设计，在保持强正确性保证的同时显著提升内存管理效率，获得 SOSP 2025 最佳论文奖[8]。

1.4 本文工作

> **【待完善】** 分 3-4 段简述本文主要工作与贡献，建议分成两层表述：
> 1. 完整兼容性内核实现：在 ArceOS 基础上实现 StarryOS，支持进程、内存、文件系统、网络、终端、信号等核心能力，并实现 200+ Linux 系统调用
> 2. 关键机制与性能优化：围绕组件化兼容内核中的关键问题，设计并实现 extern_trait、用户态切换与反转控制流、高效内核-用户交互、异步 I/O、多级缓存等机制
> 3. AI 应用支持探索：面向 AIoT 场景，探索 NPU 驱动与 AI 应用运行支持方向

1.5 论文结构安排

全文共分为七章。第 1 章介绍研究背景、问题陈述、相关工作以及本文的主要研究内容。第 2 章结合 ArceOS 的组件化基础与灵活内核理念，阐明 StarryOS 的系统定位、设计目标与总体架构。第 3 章进一步展开兼容性内核核心子系统的设计与实现，包括系统调用、任务与进程管理、内存管理、文件系统、网络与终端以及信号与同步机制。第 4 章围绕组件化兼容内核实现中的关键技术问题，介绍 extern_trait、用户态切换与反转控制流、高效内核-用户交互、异步 I/O 以及多级缓存等机制与优化。第 5 章讨论面向 AIoT 场景的 AI 应用支持探索。第 6 章从功能完整性、应用兼容性和性能等维度对系统进行实验评测。第 7 章对全文工作进行总结，并展望后续研究方向。

第 2 章 系统背景与总体架构

第 1 章已经介绍了操作系统内核架构演进、组件化操作系统理念以及 AIoT 场景对操作系统的核心需求，并指出构建兼具兼容性、组件化与高性能的内核所面临的技术挑战。本章在此基础上，进一步阐明 StarryOS 的系统定位与设计目标，并从总体分层、执行模型和核心资源对象三个层面概述系统结构，为后续各核心子系统实现与关键机制优化提供统一的框架。

2.1 设计目标

StarryOS 的首要目标，是在 ArceOS 组件化基座之上构建一个具备较强 Linux 应用兼容能力的宏内核系统，使其能够承载更丰富的用户态程序、开发工具链和典型服务程序运行。与单纯面向专用化任务的内核不同，兼容性目标要求系统在进程管理、文件系统、信号、网络、终端和事件机制等方面逐步对齐 Linux/POSIX 语义，从而使现有软件生态能够以较低成本迁移到系统之上。

在兼容性之外，StarryOS 还强调组件化复用与性能优化并重。一方面，系统需要延续 ArceOS 的组件化思想，将与具体内核形态弱相关或可抽象的能力拆分为独立模块，以降低工程耦合并支持后续开源复用；另一方面，兼容性内核在系统调用路径、用户态访存、I/O 处理和缓存管理等位置存在明显性能压力，因此需要围绕关键路径进行专门优化。进一步地，面向 AIoT 场景的后续演化还要求系统具备良好的跨架构移植能力以及接入专用硬件与 AI 应用运行时的潜力。因此，StarryOS 的总体目标可以概括为：在组件化基础上实现完整兼容性，并在此基础上持续追求工程可维护性与系统性能。

2.2 ArceOS 基础与 StarryOS 定位

2.2.1 ArceOS 的灵活内核理念

ArceOS 的设计出发点，是避免将操作系统定制工作绑定在某一种固定内核架构之上。传统定制内核通常围绕单一形态展开，当应用需求从专用化运行转向更强的隔离性或更丰富的兼容性时，已有工作往往难以直接迁移，只能重新开发或退回通用内核。为降低这一迁移成本，ArceOS 提出了灵活内核的设计原则，即将不同内核形态共享的基础服务抽象为可复用组件，而将内核最终表现出的形态视为初始

化完成后执行逻辑的结果。这样一来，同一套底层能力便可以被组织为 Unikernel、宏内核或 Hypervisor 等不同系统。

在实现路径上，ArceOS 采用组件化方法构建硬件抽象、运行时支撑和上层功能模块，并强调组件之间通过明确接口进行协作。该设计的价值不在于预先规定唯一正确的系统结构，而在于为不同内核形态保留足够的组合空间，使得上层系统能够在共享底座能力的前提下针对目标场景进行定制。对于本文所研究的 StarryOS 而言，ArceOS 提供的不仅是一个可复用的底层基座，更是一种将兼容性内核实现建立在组件化原则之上的方法论基础。

2.2.2 ArceOS 的组件层次结构

从实现组织上看，ArceOS 及其上层扩展通常可以分为若干层次。最底层是与硬件平台和执行环境直接相关的基础支撑，负责硬件抽象、系统初始化以及运行时支持；在其之上是围绕内存管理、任务调度、文件系统、网络、设备驱动等组织的核心功能模块；再向上则是一些能够在不同内核形态中复用的通用组件；最上层才是具体系统形态对这些能力的组合与使用。这样的层次划分有助于将“底层公共能力”和“上层系统策略”分离开来。

对于本文所讨论的系统而言，也可以沿着这一思路来理解其构建路径：底层复用组件化基座提供的硬件抽象与运行时能力，中间层承载内存、任务、文件系统、网络等基本操作系统功能，而上层则在此基础上实现面向 Linux 兼容性的宏内核功能。图 X 给出了总体层次结构，说明了各层之间的依赖关系。

2.2.3 StarryOS 的系统定位

StarryOS 可以看作 ArceOS 在 Linux 兼容性宏内核方向上的一条重要扩展路径。与 ArceOS 原本更强调灵活内核与组件复用的实验性基座定位不同，StarryOS 更直接地面向用户态程序运行环境，要求系统能够提供完整的进程抽象、地址空间管理、文件与设备抽象、网络接口以及信号与事件处理机制，从而承载更接近 Linux 使用习惯的软件生态。因此，StarryOS 并不是对 ArceOS 基础模块的简单堆叠，而是在复用底座能力的同时，将大量 POSIX/Linux 兼容语义逐步落实到具体实现中。

对于本文而言，更重要的并不是工程目录如何命名，而是系统在结构上如何划分职责边界。整体上看，StarryOS 在下层继承了组件化基座提供的通用能力，在上层围绕兼容性需求组织进程、内存、文件、网络和信号等核心功能，并通过若干相对独立的模块承载特定语义。这种定位既保留了组件化设计带来的可复用性，也使系统能够朝着完整兼容性内核的方向持续演化。

2.3 系统总体架构

2.3.1 总体分层与模块关系

从总体上看，StarryOS 的结构可以概括为“组件化基座 + 宏内核主体 + 若干相对独立的功能模块”三部分。底层部分负责提供硬件抽象、运行时支撑以及调度、内存、文件系统、网络等基础能力；中间的宏内核主体负责组织用户程序加载、系统调用分发以及多个子系统之间的协作；上层则围绕兼容性需求实现进程管理、信号处理、虚拟内存访问等具体功能。这样的组织方式使系统既能复用底层能力，又能在主体部分集中实现 Linux 兼容性相关逻辑。

从功能关系上看，系统调用层位于用户态与内核态交互路径的中心位置，负责将陷入内核的请求分派到任务管理、内存管理、文件系统、网络与信号等子系统；文件系统和网络子系统又分别通过统一的对象抽象与设备、缓存和事件机制发生关联；任务管理与调度机制则为整个系统提供执行上下文；信号、poll/epoll、futex 等同步与事件相关机制横向贯穿多个子系统，构成阻塞、唤醒和中断语义的统一基础。后续章节将沿着这些模块边界，分别展开对核心子系统和关键机制的介绍。

2.3.2 组件边界与仓库组织

StarryOS 的工程组织延续了 ArceOS 的组件化拆分思路，完整系统包含 30 余个相关代码仓库，分别面向底层基础能力、宏内核特有功能以及通用工具抽象。这种组织方式通过接口隔离来控制依赖方向，使不同层次的功能能够在尽可能小的影响范围内迭代。

在仓库组织上，系统遵循“与具体系统形态弱相关的能力尽量独立、与宏内核实现强绑定的逻辑集中维护”的原则。本文后续在描述系统结构时，重点放在功能边界划分与依赖关系控制上。

2.4 内核执行模型

2.4.1 用户态与内核态的基本切换流程

在 StarryOS 中，用户态程序与内核态之间的切换主要通过系统调用、异常和中断三种途径触发。当用户程序执行陷入指令（如 x86_64 的 `syscall` 或 RISC-V 的 `ecall`）时，处理器硬件自动完成特权级切换、保存用户上下文并跳转到预定义的内核入口点。内核入口代码负责进一步保存完整的用户态寄存器状态，构建内核栈帧，随后进入统一的系统调用分发路径。

系统调用分发由 handle_syscall 函数统一处理。该函数首先从用户上下文中提取系统调用号，通过 Sysno 类型进行合法性校验，随后根据调用号将请求分派到对应子系统处理函数。例如，文件相关系统调用（openat、read、write 等）被路由到文件系统模块，进程管理相关调用（clone、execve、wait4 等）被路由到任务管理模块。每个系统调用处理函数完成具体逻辑后返回结果，分发函数将返回值写入用户上下文的返回值字段，最后通过异常返回指令恢复用户态执行。

对于异常和中断，处理流程与系统调用类似但入口点不同。页异常触发地址空间的缺页处理逻辑，由 AddrSpace::handle_page_fault 根据映射后端类型完成懒分配或写时复制操作；外设中断则通过中断控制器分发到对应设备驱动处理。整个陷入-处理-返回路径构成了内核执行的基础骨架，后续介绍的进程管理、内存管理和事件机制都围绕这一路径展开。

2.4.2 进程、线程与任务对象关系

StarryOS 的执行实体抽象建立在 ArceOS 的任务（Task）概念之上，同时引入进程（Process）和线程（Thread）两层结构以对齐 Linux 语义。在 ArceOS 中，任务是最基本的调度单位，由 axtask 模块管理，包含任务标识符、状态、调度优先级、内核栈和任务扩展（TaskExt）等核心属性。StarryOS 并不直接修改 ArceOS 的任务定义，而是通过 extern_trait 机制将自定义的 Thread 结构挂接到任务扩展字段中，从而在不侵入底层调度器的前提下承载 Linux 兼容性所需的额外状态。

进程（Process）由独立的 starry_process crate 定义，是进程关系和线程组的容器。每个进程包含进程标识符（PID）、线程组、父进程引用和子进程集合、所属进程组和会话等核心属性。进程组（ProcessGroup）和会话（Session）实现了作业控制和终端管理所需的层级关系。为支持 POSIX 语义中进程组和会话的隐式创建与回收，系统设计了反向存储结构：进程持有进程组的强引用，而进程组持有进程的弱引用表。该弱引用表由专门开发的 weak-map crate 实现，在进程退出时自动失效，无需显式清理。

线程（Thread）由内核模块定义，持有对进程数据的引用，同时维护线程级信号管理器、定时器状态等线程特有属性。每个线程通过 Box<Thread> 形式存储在任务扩展中，实现了从 ArceOS 任务到 StarryOS 线程的无缝桥接。进程数据是进程级资源的集中管理点，包括地址空间、文件描述符表、命令行参数、信号处理器表等。这种“进程共享、线程独立”的资源划分方式与 Linux 的进程/线程模型高度一致。

2.5 核心资源管理对象

在介绍完执行模型之后，本节从资源对象角度概括 StarryOS 的组织方式。兼容性内核的核心职责是将用户程序看到的抽象对象（进程、文件、地址空间等）映射到内核内部的具体数据结构，并保证这些对象在生命周期管理、并发访问和跨子系统协作中的一致性。后续各子系统实现章节将围绕这些对象展开，此处先给出整体视角。

2.5.1 地址空间与内存对象

地址空间（`AddrSpace`）是 StarryOS 虚拟内存管理的核心对象，封装了虚拟地址范围、内存区域集合（`MemorySet`）和页表（`PageTable`）三部分。每个进程拥有独立的地址空间实例，由 `ProcessData` 持有并通过互斥锁保护。地址空间支持映射、取消映射、权限修改和缺页处理等基本操作，其内部通过统一的 Backend 抽象支持不同类型的映射后端，包括线性映射、按需分配、写时复制和文件映射等。

内存区域（`MemoryArea`）描述一段连续虚拟地址的属性，包括起始地址、大小、权限标志和后端类型。后端（`Backend`）决定了该区域的物理页分配策略：`LinearBackend` 用于固定物理地址映射，`CowBackend` 支持写时复制的按需分配，`FileBackend` 将文件内容映射到内存并支持页缓存，`SharedBackend` 用于跨进程共享内存。这种分层设计使 `mmap`、`brk` 等系统调用能够统一处理匿名映射、文件映射和共享映射，同时为后续缓存优化提供清晰的承载点。

2.5.2 文件、设备与 I/O 对象

StarryOS 采用类 Unix 的文件抽象模型，通过 `FileLike` trait 统一定义文件、管道、套接字、事件 fd 和信号 fd 等可读写对象的行为。`FileLike` 提供读取、写入、状态查询、ioctl 控制和非阻塞标志设置等接口，同时继承 `Pollable` trait 以支持事件等待机制。每个打开的文件由 `FileDescriptor` 结构描述，包含指向具体 `FileLike` 实现的引用以及 close-on-exec 标志。文件描述符表通过 `Scope` 机制管理，支持按进程隔离以及 `clone` 时共享或复制。

普通文件由 `File` 结构表示，内部持有对文件系统后端（`FileBackend`）的引用，后者可能是页缓存或直接设备访问。设备和伪文件系统通过 VFS 层纳入统一访问框架，`Device` trait 提供设备特有的 mmap 支持和 ioctl 处理。套接字由 `Socket` 结构表示，封装了 TCP、UDP 和 Unix Domain Socket 等网络通信能力。管道、事件 fd 和信号 fd 分别由 `Pipe`、`EventFd` 和 `Signalfd` 实现，均遵循 `FileLike` 接口。

这种统一抽象使 `read`、`write`、`poll`、`epoll_ctl` 等系统调用能够跨不同对象类型工作，简化了事件驱动和 I/O 多路复用的实现。

2.5.3 通信与事件对象

进程间通信和事件驱动机制在 StarryOS 中由 socket、poller、waker 和 signal 等对象共同支撑。套接字（`Socket`）封装网络协议栈提供的通信能力，同时作为 `FileLike` 对象可被纳入文件描述符表并通过 epoll 监控。事件等待机制由 `PollSet`、`Pollable` 和 `Waker` 三层构成，为 poll/select/epoll、futex 等待、子进程退出等待等场景提供统一基础。

信号机制由独立的 `starry_signal` crate 实现，采用进程级和线程级两层管理结构。`ProcessSignalManager` 维护进程级待处理信号队列和信号处理器表，`ThreadSignalManager` 维护线程级待处理队列、信号掩码和信号栈。信号投递时根据目标选择进程或线程队列，并在系统调用返回路径检查和处理。

2.6 本章小结

本章基于第 1 章提出的问题背景，进一步阐明了 StarryOS 的设计目标与系统定位。系统以 Linux 应用兼容性为核心目标，在 ArceOS 组件化基座之上构建宏内核形态，同时兼顾组件化复用与关键路径性能优化。本章还从总体分层、执行模型和核心资源对象三个维度概述了系统结构，为后续各核心子系统实现与关键机制优化提供了整体框架。

第 3 章 核心子系统设计与实现

本章在第 2 章总体架构的基础上，进一步介绍 StarryOS 兼容性内核中若干核心子系统的组织方式与实现基础。与第 4 章聚焦关键机制和性能优化不同，本章更关注系统作为一个完整 Linux 兼容性内核所必须具备的“基本盘”，即系统如何承载用户程序的执行、如何管理地址空间与文件对象、如何处理网络与终端输入输出，以及如何通过信号与同步机制组织进程间交互。通过这些内容，可以更清晰地理解后续各项优化工作是建立在怎样的系统基础之上的。

3.1 系统调用框架

系统调用框架是 StarryOS 连接用户态程序与内核子系统的核心入口。当用户程序通过陷入指令进入内核后，系统首先根据处理器提供的上下文信息识别系统调用号及参数，随后进入统一的系统调用分发路径，再由内核将请求转交给文件、内存、任务、时间、信号等具体子系统处理，最后将返回值写回用户上下文并恢复执行。从设计目标上看，该框架既要尽可能贴近 Linux 的系统调用语义，又要避免将过多具体逻辑堆叠在单一路径中，从而保持模块边界清晰。

系统调用相关逻辑主要集中在 `starry-kernel` 的 `syscall` 模块下，采用“统一入口、按功能域分解实现”的组织方式：对外保持一致的系统调用进入路径，对内则按照资源、信号、时间等主题划分子模块，减少不同子系统之间的直接耦合。这种结构有利于逐步补齐系统调用支持范围，并在后续演化中持续维护语义一致性。

3.2 任务与进程管理

任务与进程管理子系统负责用户程序的创建、执行、终止和状态查询，是实现 Linux 进程模型的核心。该子系统围绕 `clone`/`fork`、`execve`、`wait4` 和 `exit` 四类系统调用展开，同时在实现上与 ArceOS 的任务调度框架紧密协作。

进程创建通过 `clone` 系统调用实现，该调用接收一组标志位控制父子进程间的资源共享行为。`CLONE_VM` 标志使子进程共享父进程的地址空间，对应线程创建场景；`CLONE_FILES` 和 `CLONE_FS` 标志分别控制文件描述符表和文件系统信息（当前目录、根目录）的共享；`CLONE_SIGHAND` 控制信号处理器表的共享；`CLONE_THREAD` 将新线程加入父进程的线程组。`fork` 系统调用在 x86_64 架构上实现为 `clone(SIGCHLD)` 的简化形式。实现上，`CloneArgs::do_clone` 方法负责参数

校验、新任务创建、地址空间克隆（通过 AddrSpace::try_clone 实现 CoW）、资源复制或共享、新线程挂接到进程，最后将任务加入调度队列。

程序加载与替换由 execve 系统调用处理。该调用接收可执行文件路径、命令行参数和环境变量，负责清空当前地址空间（保留 CoW 映射的原始页）、加载 ELF 文件、设置新的入口点和栈顶地址、更新进程元数据（路径、命令行）、重置信号处理器为默认值、关闭带 close-on-exec 标志的文件描述符，最后修改用户上下文以跳转到新程序入口。execve 保持进程标识符和线程不变，仅替换地址空间内容，这与 Linux 的语义一致。

进程对象由独立的 starry_process crate 定义，内部包含线程组（ThreadGroup）结构。线程组记录属于同一进程的所有线程 ID 集合、进程退出码以及组退出标志（group_exited）。当调用 exit_group 系统调用时，组退出标志被设置，表示整个进程正在终止。进程还维护父进程的弱引用和子进程的强引用集合，支持 wait4 系统调用查询子进程状态。

进程组和会话管理是作业控制的基础。setsid 系统调用创建新会话并使调用进程成为会话首进程和进程组首进程，setpgid 将进程加入指定进程组。会话-进程组-进程构成一个树状层级结构，但与常规树状存储不同，POSIX 语义中进程组和会话是隐式创建和回收的：进程调用 setsid 时隐式创建会话和进程组，进程退出时隐式从所属容器中移除。为支持这种隐式生命周期管理，系统设计了反向存储结构：进程持有进程组的强引用（Arc<ProcessGroup>），进程组持有进程的弱引用表（WeakMap<Pid, Weak<Process>>）。当进程退出时，其弱引用自动失效，无需显式调用清理操作。

为支持这一设计，系统专门开发了 weak-map crate。该 crate 基于 BTreeMap 实现弱引用映射，提供 WeakMap 和 StrongMap 两种类型。WeakMap 存储弱引用，在迭代访问时自动跳过已失效的条目；同时实现了惰性清理机制，在操作计数达到阈值时自动移除过期条目。这一设计避免了显式生命周期管理的复杂性，使进程组和会话的管理代码更加简洁。

线程对象由内核模块中的 Thread 结构定义，持有对进程数据（ProcessData）的共享引用。线程维护线程级信号管理器（ThreadSignalManager）、定时器状态（TimeManager）、退出标志（exit）以及 robust futex 列表等线程特有属性。每个线程通过 Box<Thread> 形式存储在 ArceOS 任务扩展（AxTaskExt）中，通过 extern_trait 机制实现 on_enter 和 on_leave 回调，在任务切换时自动激活和切换进程资源作用域。

进程数据是进程级资源的集中管理点，包括地址空间（AddrSpace）、文件描述符表（通过 Scope 管理）、命令行参数、信号处理器表、futex 表以及进程退出事件

等。所有属于同一进程的线程共享同一 ProcessData 实例，而不同进程的线程则拥有独立的 ProcessData。这种“进程共享、线程独立”的资源划分方式与 Linux 的进程/线程模型高度一致，也是实现 clone 系统调用中各种共享标志语义的基础。

进程终止通过 exit 和 exit_group 系统调用实现。前者终止当前线程，后者终止整个进程的所有线程。实现上，线程退出时从线程组中移除自身并记录退出码，当最后一个线程退出时触发进程退出。进程退出时标记为僵尸状态，其子进程被 init 进程接管，父进程通过 wait4 或 waitpid 系统调用查询退出状态并释放僵尸进程资源。wait 实现采用异步等待模式，通过 block_on 和 interruptible 包装使等待过程可被信号打断，符合 POSIX 的可中断系统调用语义。

3.3 内存管理

内存管理子系统负责用户进程地址空间的创建、映射、扩展和回收，是兼容性内核中与硬件交互最频繁的部分。StarryOS 的内存管理建立在 AddrSpace 核心对象之上，通过多种后端（Backend）类型支持不同映射场景，同时实现了按需分配、写时复制等关键优化机制。

brk 系统调用用于扩展或收缩进程堆区域。实现上，系统维护一个 heap_top 变量记录当前堆顶地址，新分配请求在预定义的堆范围内按页对齐扩展映射。brk 不立即分配物理页，而是仅建立虚拟到物理的映射关系，实际物理分配在首次访问时通过缺页处理完成。这种懒分配策略减少了启动时的内存占用，同时保证了与 Linux brk 语义的兼容。

mmap 系统调用是更通用的内存映射入口，支持匿名映射、文件映射和共享映射等多种类型。匿名映射（MAP_ANONYMOUS）使用 CowBackend 后端，按需分配物理页并支持写时复制；私有文件映射同样使用 CowBackend，但物理页从文件页缓存获取，首次写入时触发 CoW 分配新页；共享文件映射使用 FileBackend，直接与页缓存关联，写入同步到文件；共享匿名映射使用 SharedBackend，通过 SharedPages 结构支持跨进程共享。mmap 还支持 MAP_FIXED、MAP_POPULATE 等标志控制地址选择和预分配行为。

写时复制（Copy-on-Write, CoW）是内存管理的关键优化机制，主要用于 fork 后的地址空间克隆。当父进程 fork 子进程时，AddrSpace::try_clone 方法遍历所有内存区域，为新地址空间创建共享同一物理页的映射，并将双方映射的权限改为只读。当任一方首次写入某页时，缺页处理逻辑分配新物理页、复制原页内容、更新映射指向新页并恢复写权限。这一机制使 fork 代价从复制全部内存降低到仅建立映射关系，大幅提升了进程创建性能。

缺页处理由 `AddrSpace::handle_page_fault` 实现，根据发生缺页的地址找到对应内存区域，调用后端的 `populate` 方法分配物理页并建立映射。对于 `CowBackend`，populate 检查是否需要复制原页；对于 `FileBackend`，populate 从页缓存读取文件内容。缺页处理完成后，用户程序可继续执行，整个过程对用户态完全透明。这一懒分配-缺页处理的组合构成了现代操作系统内存管理的基本范式，StarryOS 的实现沿用了这一设计并在此基础上进一步优化。

3.4 文件系统

StarryOS 的文件系统设计遵循类 Unix 系统常见的 VFS 分层思路，即通过统一抽象将具体物理文件系统、伪文件系统以及设备文件纳入同一访问框架。在这一框架下，文件描述符负责向进程暴露统一的访问句柄，`file` 对象承载一次打开后的状态，`inode` 与目录项等对象则承担对底层命名空间和存储实体的组织。这种设计的意义在于，将进程看到的文件操作接口与具体文件系统实现解耦，从而能够在相同系统调用语义下支持普通磁盘文件、管道、套接字以及 `/proc`、`/dev` 等特殊文件。

StarryOS 既需要处理普通文件系统，也需要实现 `procfs`、`tmpfs`、`devfs` 等伪文件系统支持。因此，VFS 层不仅是功能复用的基础，也是后续缓存优化的重要承载点。页缓存、块缓存和目录项缓存等机制虽然将在后文重点讨论，但它们能够发挥作用的前提，正是系统已经建立起统一的文件对象与路径解析框架，使不同来源的数据访问都可以被纳入一致的管理路径中。

3.5 网络与终端子系统

网络与终端子系统共同构成了 StarryOS 面向真实用户空间应用的重要外部交互能力。系统网络栈建立在 `smoltcp` 之上，并在其外封装 POSIX 风格的 socket 抽象，以支持 TCP、UDP 和 Unix Domain Socket 等常见通信方式；同时，系统还逐步补齐了多设备接入和与事件机制协同工作的能力。网络相关逻辑分布在文件抽象和底层网络模块的结合处，反映出“套接字即文件对象”的 Unix 风格设计思路。

终端相关能力则主要面向交互式应用和开发工具链支持。为了运行 shell、编辑器以及更复杂的图形与文本应用，系统需要提供 tty/pty、行规程以及作业控制等基础机制，使应用能够在接近 Linux 的终端语义下运行。这些能力本身未必体现出特别强的创新性，但它们对于兼容性内核的完整性至关重要，也是后续展示 Alpine Linux、开发工具链和图形环境支持时不可缺少的系统基础。

3.6 信号与同步机制

信号与同步机制贯穿于 StarryOS 的多个子系统之中，是实现类 Linux 进程交互语义的重要支撑。信号机制由独立的 starry_signal crate 实现，负责在异步事件发生时向目标进程或线程传递通知，并要求系统在合适的边界上检查待处理信号、构造用户态处理上下文并恢复执行；同步机制则需要为进程阻塞、等待和唤醒提供统一基础，包括 futex、poll/select/epoll 等常见设施。

信号机制采用进程级和线程级两层管理结构。ProcessSignalManager 维护进程级待处理信号队列（PendingSignals）、信号处理器表（SignalActions）以及所有线程信号管理器的弱引用列表。信号处理器表是一个 64 元素的数组，每个元素记录对应信号的处理器地址、掩码和标志（如 SA_RESTART、SA_SIGINFO 等）。ThreadSignalManager 维护线程级待处理队列、信号掩码（blocked）和信号栈（stack），并在构造时自动注册到进程管理器的弱引用列表中。当线程退出时，其弱引用自动失效，无需显式注销。

待处理信号结构（PendingSignals）区分标准信号和实时信号的处理方式。标准信号（1－31）每种最多一个实例待处理，使用固定大小的 Option<Box<SignalInfo>> 数组存储，后续发送的相同信号被丢弃。实时信号（32－64）可排队多个实例，使用 VecDeque<SignalInfo> 数组存储，保证不丢失。信号集（SignalSet）使用位图记录当前待处理的信号编号，支持高效的集合操作。信号出队时根据掩码选择编号最小的待处理信号，并自动更新信号集。

信号投递分为进程级和线程级两种路径。当向进程发送信号时（如 kill 系统调用），ProcessSignalManager::send_signal 方法首先检查该信号是否被忽略，若未被忽略则将信号加入进程队列，随后遍历线程弱引用列表找到第一个未阻塞该信号的线程进行唤醒。若所有线程都阻塞该信号，则信号保留在进程队列中等待解禁。当向线程发送信号时（如 tgkill 系统调用），ThreadSignalManager::send_signal 方法直接将信号加入线程队列并唤醒线程。

为优化信号检查性能，进程级和线程级管理器分别维护 possibly_has_signal 原子布尔变量。当信号入队时设置为 true，当信号出队且队列为空时设置为 false。在系统调用返回路径检查信号时，ThreadSignalManager::check_signals 方法首先读取该变量进行快速判断：若两个变量都为 false，则直接返回不进行任何处理；仅当至少一个为 true 时才进入完整的出队和处理流程。这一设计避免了每次系统调用返回时都进行锁操作和队列检查的开销。

信号处理时需构造用户态信号帧（SignalFrame），包含保存的用户上下文（UContext）、信号信息（SignalInfo）和原用户上下文（UserContext）。帧被写入用户信号栈，随后用户上下文的指令指针被设置为用户定义的处理器地址，栈指针指向信号帧，参数寄存器设置为信号编号、信号信息指针和上下文指针。处理器返回地址被设置为信号蹦床（signal trampoline）代码，该代码在系统初始化时复制到用户地址空间的固定位置，负责调用 rt_sigreturn 系统调用恢复原有执行流。

futex 机制为用户态同步原语提供内核级等待和唤醒能力。每个进程维护一个 futex 表，以用户地址为键存储等待队列（WaitQueue）。FUTEX_WAIT 操作检查用户地址的值，若匹配则阻塞当前线程；FUTEX_WAKE 操作唤醒指定数量的等待者。futex 与信号机制的交互体现在等待过程可被信号打断，此时系统调用返回错误码 EINTR，用户态需要据此决定是否重试或执行其他逻辑。若信号处理器设置了 SA_RESTART 标志，内核会自动重启被中断的系统调用。

从系统组织角度看，信号机制并不是一个孤立模块，而是与系统调用返回路径、任务状态切换以及 I/O 事件处理紧密结合。特别是在阻塞系统调用场景下，信号能否正确打断等待过程、系统是否能够按照预期返回错误码或执行重启语义，直接决定了用户态应用的兼容性表现。starry_signal 作为独立 crate 设计，将信号管理逻辑与具体内核实现解耦，便于测试和复用。

3.7 本章小结

本章介绍了 StarryOS 作为 Linux 兼容性内核所必须具备的核心子系统，包括系统调用框架、任务与进程管理、内存管理、文件系统、网络与终端以及信号与同步机制。这些子系统构成了系统的“基本盘”，使 StarryOS 能够承载用户态程序加载、多进程并发执行、文件和网络 I/O 以及进程间通信等典型应用场景。

从实现角度看，这些子系统的共同特点是在复用 ArceOS 底层能力的基础上补充 Linux 兼容语义。任务调度、等待队列、文件系统框架和网络栈由 ArceOS 或其依赖模块提供，StarryOS 通过 extern_trait 机制扩展任务状态、通过 ProcessData 组织进程级资源、通过统一的 FileLike 抽象整合各类 I/O 对象。这种分层协作的设计使系统在保持组件化架构的同时，逐步对齐 POSIX/Linux 的复杂语义要求。

值得注意的是，部分子系统被抽取为独立 crate 以提高复用性。starry_process 实现进程关系管理，包括进程组、会话和线程组；starry_signal 实现信号机制，区分进程级和线程级管理；weak-map 提供弱引用映射，支持会话-进程组-进程层级结构的隐式生命周期管理。这些 crate 可以被其他需要类似功能的系统直接复用，体现了组件化设计在工程实践中的价值。

这些基础子系统虽然本身未必体现强烈的创新性，但它们为后续关键机制与性能优化工作提供了必要的承载平台。用户态访存优化、异步 I/O 机制、多级缓存设计以及信号与事件等待的配合，都需要建立在完整且语义正确的子系统之上。下一章将聚焦这些影响系统结构和性能上限的关键机制，说明 StarryOS 如何在组件化实现与性能优化之间取得平衡。

第 4 章 关键机制与性能优化

在完成兼容性内核基本功能之后，StarryOS 面临的主要问题不再只是“是否具备某项能力”，而是“这些能力能否以可维护、可复用且足够高效的方式组织起来”。一方面，ArceOS/StarryOS 所坚持的组件化路线要求系统在 Rust 严格的依赖与类型约束下处理复杂的跨模块协作；另一方面，Linux 兼容性内核在用户态访存、I/O 路径、事件等待与缓存管理上天然存在显著的性能压力。因此，本章聚焦于若干真正影响系统结构和性能上限的关键机制，说明系统在组件化实现与性能优化之间所做的设计取舍。

4.1 Rust 组件化开发的挑战

Rust 的模块与包管理机制天然强调依赖关系在编译期的静态确定，这使其在构建大型系统时能够获得良好的类型安全与优化效果，但也会给组件化操作系统开发带来特殊挑战。对于内核而言，某些功能依赖并不总是沿着“底层依赖上层”的单向路径展开。例如，底层组件有时需要回调上层提供的能力，或者在抽象数据结构中挂接由上层定义的扩展状态；再如 `#[panic_handler]`、`#[global_allocator]` 这类入口级能力，往往天然表现出一种“反向依赖”的特征。与 C 语言更多在链接阶段解决符号关系不同，Rust 需要在更早阶段就将这些依赖明确下来，因此会更早暴露出组件边界不清或依赖方向不合理的问题。

这一问题在操作系统开发中尤为突出。操作系统内部模块数量多、层次深，而且控制流和资源关系往往比普通应用程序更复杂，简单地通过重新组织调用顺序并不能完全消除循环依赖或跨层扩展需求。因此，如何在不破坏 Rust 类型系统和性能优势的前提下，为上层模块向下层注入实现、为下层模块保留可扩展接口，就成为组件化宏内核设计中的一个核心问题。后文介绍的 `extern_trait` 机制，正是在这一背景下提出的。

4.2 extern_trait 机制

4.2.1 设计动机

在组件化操作系统开发中，跨 crate 的接口定义与实现解耦是一个核心需求。ArceOS 项目早期引入的 `crate_interface` 机制正是为解决这一问题而设计：它允许在一个 crate 中定义 trait 接口，在另一个 crate 中提供实现，然后通过链接器符号

在运行时连接。这种方式成功避免了循环依赖，使底层模块能够调用上层提供的能力，例如日志模块调用平台层提供的 console_write_str 函数。

然而，crate_interface 的设计存在若干局限性。首先，它不支持带接收者参数的方法（&self、&mut self），只能定义关联函数，因此无法表达具有状态的接口。更重要的是，它无法自动处理 RAII 语义——当实现类型需要资源清理时，调用方必须显式调用某种 drop 方法，这不仅破坏了 Rust 的所有权语义，也增加了使用复杂度。

面对这些局限，常见的替代方案也存在明显缺陷。Box<dyn Trait> 虽然能完整表达 trait 语义，但需要堆分配每个 trait 对象，且每次方法调用都要经过 vtable 进行间接寻址，在内核性能敏感路径上代价过高。另一种极端方案是使用 *mut () 进行手动类型擦除，这虽然避免了堆分配和间接调用，但完全依赖 unsafe 代码进行类型转换，极易引入内存安全漏洞。这些方案要么牺牲性能，要么牺牲安全性，难以满足操作系统内核的需求。

值得注意的是，Rust 语言团队正在推进的 EII（Externally Implementable Items）特性试图从语言层面解决类似问题。该特性采用属性语法，使用 #[eii(...)] 定义可外部实现的函数或静态变量，使用同名属性提供实现，类似于 #[panic_handler] 和 #[global_allocator] 的机制。但这一特性的 RFC 尚未正式合并，目前仅处于实验阶段（#![feature(extern_item_impls)]），且仅支持函数而非完整的 trait 语义。因此，StarryOS 需要一种既能获得完整 trait 语义，又不依赖 nightly 特性的解决方案。

extern_trait 机制正是在这一背景下设计的。它的核心目标是：在不依赖语言层面修改的前提下，通过过程宏实现零开销的跨 crate trait 调用，支持带接收者的方法、自动 RAII 处理，并保持完整的类型安全。与 Rust 官方的 EII 特性相比，它作为第三方库可在 stable Rust 上使用，并且提供更完整的 trait 支持；与传统的动态分派方案相比，它在编译期完成所有决议，在链接时绑定具体实现，避免了任何运行时开销。

4.2.2 实现原理

extern_trait 的实现基于一个核心观察：在大多数调用约定中，大小不超过两个指针的结构体可以通过寄存器直接传递，无需经过栈内存。这一观察启发了一种“静态 vtable”的设计思路——将 trait 的方法分派从运行时的间接调用转变为链接时的直接符号绑定。

具体而言，extern_trait 通过过程宏完成三方面的工作。首先，在 trait 定义侧，宏生成一个固定大小的代理类型 Repr，其大小为两个指针（16 字节在 64 位平

台，8 字节在 32 位平台），足以存储大多数常见的实现类型，如 Box<T>、Arc<T>、&T 以及胖指针 &[T]、&str 等。宏同时为该代理类型生成 trait 实现，其中每个方法都被替换为对外部符号的调用：

```rust
// 定义侧生成的代码（简化）
#[repr(transparent)]
pub struct HelloProxy(extern_trait::Repr);

impl Hello for HelloProxy {
    fn new(arg: i32) -> Self {
        extern "Rust" {
            #[link_name = "extern_trait_Hello_new"]
            fn __new(arg: i32) -> Repr;
        }
        Self(unsafe { __new(arg) })
    }

    fn hello(&self) {
        extern "Rust" {
            #[link_name = "extern_trait_Hello_hello"]
            fn __hello(this: &Repr);
        }
        unsafe { __hello(&self.0) }
    }
}

impl Drop for HelloProxy {
    fn drop(&mut self) {
        extern "Rust" {
            #[link_name = "extern_trait_Hello_drop"]
            fn __drop(this: *mut Repr);
        }
        unsafe { __drop(&mut self.0 as *mut _) }
    }
}
```

其次，在 trait 实现侧，宏生成对应的符号导出函数。每个 trait 方法都被包装为一个带有 `#[export_name]` 属性的函数，该函数接收 `Repr` 类型的参数，在其中完成实际实现类型的构造或方法调用：

```rust
// 实现侧生成的代码（简化）
struct HelloImpl(i32);

#[export_name = "extern_trait_Hello_new"]
extern "Rust" fn __new(arg: i32) -> Repr {
    Repr::from_value(HelloImpl::new(arg))
}

#[export_name = "extern_trait_Hello_hello"]
extern "Rust" fn __hello(this: &Repr) {
    let impl_ref: &HelloImpl = this.as_ref();
    impl_ref.hello();
}

#[export_name = "extern_trait_Hello_drop"]
extern "Rust" fn __drop(this: *mut Repr) {
    unsafe { ptr::drop_in_place(this.as_mut::<HelloImpl>()) };
}
```

最后，宏在编译期进行大小检查，确保实现类型能够容纳于 `Repr` 中。如果实现类型过大，编译时会触发静态断言错误，引导用户将类型包装在 `Box` 或其他指针类型中。

这种设计的关键优势在于：代理类型与实现类型之间通过链接器符号进行连接，而非通过运行时的 vtable。当调用方调用 `HelloProxy::hello()` 时，编译器生成的代码直接跳转到 `__hello` 符号，该符号在链接时被解析为实现 crate 中导出的函数地址。这消除了动态分派的开销，同时保持了完整的类型安全——所有类型转换都在宏生成的代码中完成，用户代码无需使用 unsafe。

对于 supertrait 的处理，宏会自动为代理类型生成相应的 trait 实现。例如，如果 trait 定义为 `trait Resource: Send + Sync + Clone + Debug`，则 `ResourceProxy` 会自动获得 `Send`、`Sync`、`Clone` 和 `Debug` 的实现，这些实现同样通过外部符号绑定，确保实现 crate 中提供的 supertrait 实现被正确传递。

4.2.3 应用场景

extern_trait 机制在 ArceOS 和 StarryOS 中有着广泛的应用，其中最具代表性的场景是任务扩展机制（TaskExt）。

在 ArceOS 的调度器模块 axtask 中，TaskExt trait 被定义为任务切换时的回调接口：

```rust
#[extern_trait(pub AxTaskExt)]
pub trait TaskExt {
    fn on_enter(&self) {}
    fn on_leave(&self) {}
}
```

这一设计允许上层模块（如 StarryOS 的进程管理）向下层模块（调度器）注入任务切换时的自定义逻辑。在 StarryOS 中，TaskExt 被实现为 Box<Thread>，用于管理进程相关的上下文切换：

```rust
#[extern_trait]
impl TaskExt for Box<Thread> {
    fn on_enter(&self) {
        let scope = self.proc_data.scope.read();
        unsafe { ActiveScope::set(&scope) };
        core::mem::forget(scope);
    }

    fn on_leave(&self) {
        ActiveScope::set_global();
        unsafe
        { self.proc_data.scope.force_read_decrement() };
    }
}
```

通过这一机制，当调度器切换任务时，会自动调用 on_enter 和 on_leave 方法，完成进程资源作用域的切换。调度器模块无需了解进程管理的具体实现，只需在 TaskInner 结构中存储 Option<AxTaskExt> 字段，并在任务切换时调用相应方法即可。这种设计实现了底层调度逻辑与上层进程管理的完全解耦。

另一个重要应用是用户态内存访问接口 VmIo。在 StarryOS 的架构中，内存管理与内核核心高度相关，但信号处理等组件（starry-signal）也需要访问用户内

存。为避免循环依赖，同时为后续的异常修复机制提供统一入口，引入了 VmIo trait。通过 extern_trait，Vm 类型可以作为跨模块的内存访问接口被传递和调用：

```rust
#[extern_trait]
unsafe impl VmIo for Vm {
    fn new() -> Self { Self(IrqSave::new()) }
    fn read(&mut self, start: usize, buf: &mut [MaybeUninit<u8>]) -> VmResult { ... }
    fn write(&mut self, start: usize, buf: &[u8]) -> VmResult { ... }
}
```

这使得文件系统、驱动、信号处理等模块可以在不依赖具体内存管理实现的情况下，通过统一的接口访问用户态数据，同时也为后续介绍的异常修复机制提供了接口层面的支撑。

目前，extern_trait 已作为独立 crate 发布于 crates.io[9]，由本文作者开发维护。其设计灵感来源于 crate_interface 机制，但在功能完整性、类型安全和性能方面均有显著改进。随着 Rust 语言层面 EII 特性的推进，未来可能存在语言原生支持与库层实现之间的融合或迁移，但 extern_trait 在当下已为组件化操作系统开发提供了一种切实可用的解决方案。

4.3 用户态切换与反转控制流

4.3.1 传统宏内核控制流的局限

传统宏内核通常将系统执行过程组织为“用户程序调用内核服务”的形式：用户态程序通过系统调用主动进入内核，内核在完成请求后再返回用户态继续执行。这种模式在概念上直观、实现上成熟，也是大多数经典操作系统教材默认采用的控制流描述方式。然而，当系统希望将更多控制流相关逻辑抽象为可复用组件时，这种结构会暴露出局限性。原因在于，内核虽然能够在陷入时临时接管控制权，却很难以统一方式主动组织用户程序执行的不同阶段，因此很多与进入用户态、离开用户态、任务切换相关的逻辑容易分散在不同位置，增加模块间耦合。

对于强调组件化的 StarryOS 而言，这种局限尤其明显。如果内核只能被动响应用户程序的调用，那么一些原本应当由统一机制处理的控制流操作，往往需要通过回调、全局状态或额外约定散落在不同模块中，不利于形成清晰边界。也正因为如此，StarryOS 在用户态切换机制上采取了更强调内核主动组织控制流的设计思路，从而为任务扩展、资源切换和后续的上下文管理提供更整洁的承载方式。

4.3.2 反转控制流设计

StarryOS 采用了一种与传统宏内核不同的控制流组织方式。在传统模型中，用户程序通过系统调用主动进入内核，内核被动响应请求后返回用户态继续执行。这种被动响应模式虽然直观，但难以将用户态切换、任务调度、信号处理等逻辑组织为可复用的组件——它们往往分散在异常处理入口、系统调用路径、调度器等多个位置，通过隐式约定相互协调。

StarryOS 的设计则将控制权反转：内核代码主动“调用”用户程序，而非被动等待用户程序的调用。具体而言，每个用户任务的入口函数并非用户程序的 `main` 函数，而是一个内核循环，其核心结构如下：

```rust
while !thread.pending_exit() {
    let reason = user_context.run();  // 进入用户态

    match reason {
        ReturnReason::Syscall => handle_syscall(&mut user_context),
        ReturnReason::PageFault(addr, flags) => handle_page_fault(...),
        ReturnReason::Exception(info) => handle_exception(...),
        ReturnReason::Interrupt => {}
    }

    check_signals(&thread, &mut user_context);  // 处理信号
}
```

`user_context.run()` 是整个设计的关键：它将处理器从内核态切换到用户态，开始执行用户代码，直到发生系统调用、异常或中断时返回。返回后，内核在一个集中的位置处理各类事件，然后再次调用 `run()` 继续用户态执行。这种设计使内核能够以统一的方式组织控制流，而非在多个入口点分散处理。

这一设计思路与 Asterinas 项目[10] 的 `UserMode::execute()` 机制相似。Asterinas 同样采用内核主动调用用户程序的模式，其任务入口函数中包含一个循环，在每次用户态返回后处理系统调用、异常和信号。两个项目的设计都体现了将控制流逻辑集中化的思路，这为组件化架构提供了便利：信号处理、地址空间切换、FPU 状态管理等逻辑可以通过调度器回调（如前文所述的 `TaskExt::on_enter/on_leave`）注入到任务切换路径中，而非散落在各个异常处理入口。

4.3.3 基于 axcpu 的实现

axcpu 组件负责封装用户态与内核态切换的底层机制。它为每种支持的架构（RISC-V、x86_64、AArch64、LoongArch64）提供了一致的接口，主要包括：

- `UserContext`：封装用户态的寄存器状态，包括通用寄存器、程序计数器、栈指针以及架构特定的状态（如 RISC-V 的 `sstatus`、x86_64 的段寄存器基址）
- `run()` 方法：执行用户态切换，返回 `ReturnReason` 枚举指示返回原因
- `TrapFrame`：异常帧结构，保存陷入内核时的完整寄存器状态

核心的用户态进入逻辑由汇编代码实现。以 RISC-V 为例，`enter_user` 函数的执行流程如下：

```
enter_user:
    # 保存内核态 callee-saved 寄存器
    addi    sp, sp, -16 * XLENB
    STR     s0, sp, 0
    ...
    STR     ra, sp, 12

    # 设置用户态入口
    csrw    sepc, t0            # 设置返回地址
    csrw    sstatus, t1         # 设置状态寄存器（SPP=0 表示返回用户态）

    # 恢复用户态寄存器
    POP_GENERAL_REGS
    LDR     sp, sp, 2           # 切换到用户态栈

    sret                        # 返回用户态
```

`sret` 指令将处理器从 S 模式切换到 U 模式，开始执行用户程序。当用户程序执行系统调用（`ecall` 指令）或发生异常时，处理器重新陷入 S 模式，跳转到 trap 向量入口。trap 处理代码根据 `scause` 寄存器判断陷入原因，构造 `ReturnReason` 并返回 Rust 代码。

x86_64 架构的实现略有不同。由于 x86_64 使用 `syscall`/`sysret` 指令进行系统调用入口和返回，而非通过统一的异常入口，axcpu 需要分别处理：`syscall` 指令跳转到 MSR `LSTAR` 指定的入口点，而异常则通过 IDT 进入 trap 处理代码。此外，x86_64 需要处理 FS/GS 段寄存器基址的切换——用户态和内核态可能使用不同的

TLS 基址，这需要在进入和离开用户态时通过 `wrmsr/rdmsr` 操作 `KERNEL_GS_BASE` MSR。

axcpu 的设计目标是将这些架构差异封装在组件内部，向上层提供统一的接口。这使得 ArceOS/StarryOS 的内核代码无需关心底层的寄存器操作细节，只需调用 `UserContext::run()` 即可完成用户态切换。同时，axcpu 也提供了 `TrapFrame` 结构的访问接口，使系统调用处理代码能够读取和修改用户态寄存器（如系统调用参数和返回值），实现了跨架构的统一抽象。

4.4 高效内核-用户交互机制

4.4.1 用户内存访问机制

【待完善】说明内核访问用户内存的设计目标——零额外开销（无异常时）。介绍 Exception Fixup 机制：在汇编层面处理页异常并保证控制流不受破坏，参考 Linux 的实现思路，在没有异常发生时不存在额外开销。

4.4.2 缓冲区抽象设计

【待完善】说明传统 `read/write` 接口（`&mut [u8]`）在处理用户数据时需要额外分配与拷贝的问题。介绍 Buf/BufMut 抽象：统一内核与用户缓冲区，大部分场景实现 one-copy I/O，说明其与 Linux 设计的契合性，以及更 Rusty 的接口特点。

4.5 异步 I/O 机制

4.5.1 设计目标

在 StarryOS 中，引入异步机制的目的并不是为了追求某种编程范式上的“新颖性”，而是为了解决同步实现中广泛存在的低效轮询问题。对于串口、网络设备、输入设备以及 `futex`、`waitpid` 等需要等待事件发生的操作，如果仍然采用频繁检查状态的同步路径，系统会在 I/O 瓶颈场景下浪费大量处理器时间，并显著拉低整体吞吐量。因此，异步机制的直接目标，是在真正存在等待需求的路径上尽量减少无效轮询，使阻塞中的任务在事件发生时被精确唤醒。

不过，异步并不天然等于高性能。纯同步实现的优点在于控制流直接、上下文切换成本较低，但复杂等待场景往往难以优雅处理；纯异步实现则更容易统一事件机制，也便于复用 Rust 异步生态中的部分思路，但往往伴随更高的状态机复杂度、额外堆内存使用以及潜在性能损失。基于这一判断，StarryOS 更倾向于在整体上保

持同步任务调度框架，而在真正耗时且需要等待事件的路径上引入异步兼容层，形成同步与异步混合的设计。这也成为后续 Poller、Pollable 与 Waker 机制设计的出发点。

4.5.2 同步与异步混合架构

【待完善】介绍 Waker/Poller 机制：PollSet（无锁单向链表存储 Waker）、Pollable 接口（poll + register），以及 Poller 实现的核心逻辑。 说明如何兼容 POSIX poll 语义。可附架构图。

4.5.3 与信号和硬件的集成

【待完善】 说明异步机制与信号（可中断的系统调用）的交互， 以及外设中断（VirtIO 网卡/输入、UART）通过注册 Waker 实现硬件唤醒的机制。

4.6 多级缓存优化

【待完善】 展开说明页缓存（page cache）、块缓存（block cache）、目录项缓存（dcache）三层缓存的实现细节，以及脏页写回策略。

4.7 多核调度与虚拟内存优化方向

4.7.1 多核调度优化

【待完善】说明当前调度实现的基础（基于 ArceOS axtask）， 以及针对多核场景规划/已实施的优化（负载均衡、调度延迟等）。如工作尚未完成，说明设计方向和参考（KRR, OSDI’25）。

4.7.2 虚拟内存优化

【待完善】说明当前虚拟内存实现（VMA + 页表双层结构），以及参考 CortenMM（SOSP’25）取消独立 VMA 抽象的优化方向。 如工作尚未完成，说明设计思路。

4.8 本章小结

【待完善】总结本章的关键机制与性能优化工作，强调其在组件化兼容内核中的作用，并引出后续 AI 应用支持与实验评测。

第 5 章 AI 应用支持探索

前两章围绕兼容性内核的核心子系统与关键机制展开，使 StarryOS 具备了承载 Linux 应用的基础能力与性能优化路径。然而，AIoT 场景对操作系统的需求不止于通用兼容性，还涉及专用硬件加速器集成与 AI 推理框架运行支持。本章从这一方向出发，探讨面向 NPU 等边缘侧加速器的驱动设计思路，以及在定制化内核上支撑典型 AI 应用运行的可行性分析。需要说明的是，本章工作目前处于探索与规划阶段，部分内容尚未完全落地，更多是对后续演进方向的梳理与初步尝试。

5.1 目标平台与硬件环境

【待完善】简述目标开发板（配备 NPU 的边缘计算设备），说明需要完成的移植工作（StarryOS 跨架构支持、驱动适配）。

5.2 NPU 驱动设计

【待完善】简述 NPU 驱动开发的思路和挑战：驱动模型、与内核内存管理的交互、DMA 支持等。

5.3 AI 应用运行支持

【待完善】简述目标应用（YOLO 目标检测、dora-rs 数据流框架）的运行需求，以及 StarryOS 需要提供的支持（推理框架兼容性、实时调度等）。

5.4 当前进展与后续计划

【待完善】如果工作尚未完成，可在此说明当前进展、阶段性结果及后续计划。

第 6 章 实验评测

在完成系统架构设计、核心子系统实现与关键机制优化后，需要从实验角度验证 StarryOS 的实际表现。本章围绕功能完整性、应用兼容性与性能三个维度展开评测：首先通过操作系统大赛标准测例检验系统调用覆盖范围与语义正确性；其次在实际应用场景中验证开发工具链、图形环境及典型服务的运行情况；最后针对系统调用开销、I/O 吞吐等关键指标进行性能基准测试，并结合第 4 章的优化机制分析其效果。

6.1 实验环境

【待完善】简述测试所用的硬件平台（x86/riscv64/aarch64 等架构、QEMU 版本）和软件环境（Rust 工具链版本、Alpine Linux 镜像版本等）。

6.2 功能完整性评测

6.2.1 系统调用测例

【待完善】介绍操作系统大赛的标准测例套件，说明 StarryOS 的通过情况（可配表格，对比赛道测例通过率）。

6.2.2 应用兼容性测试

【待完善】展示在 Alpine Linux 下运行实际应用的验证结果，包括：

- 开发工具链（GCC 编译、Cargo 构建、Git 操作）
- 图形应用（X11/dwm + ImageMagick 等）
- 其他典型应用（Python、SQLite、Redis、xv6 嵌套运行等）

可用截图或描述性文字。

6.3 性能评测

【待完善】说明性能测试的方法和基准（lmbench、UnixBench 或自定义 microbenchmark）。

6.3.1 系统调用开销

【待完善】测试典型系统调用（getpid、read、write 等）的延迟， 与 Linux 基线对比。

6.3.2 I/O 性能

【待完善】测试文件读写吞吐量，体现异步机制和多级缓存优化的效果。

6.4 优化效果分析

【待完善】将第 4 章提出的关键机制与优化点和实验结果对应起来， 分析其对系统吞吐量、延迟和资源利用率的影响。

6.5 开发工作量统计

【待完善】统计整体工作量，参考 thuthesis-zyj 表 5.3 的形式： 主仓库新增代码行数、子仓库数量、commit 数量等。

6.6 本章小结

【待完善】总结实验结果，说明 StarryOS 在功能完整性、应用兼容性和性能方面的整体表现。

第 7 章 总结与展望

7.1 工作总结

【待完善】3-4 段，逐一总结各章的主要工作和成果：

1. 系统背景与总体架构：明确 StarryOS 的设计目标、系统定位与整体结构
2. 核心子系统实现：完成兼容性内核的基本盘，实现 200+ Linux 系统调用并支撑 Alpine 生态
3. 关键机制与性能优化：组件化设计探索（extern_trait、用户态切换）以及访存、 缓冲区、异步 I/O、多级缓存等优化
4. AI 应用支持探索：面向 AIoT 场景的 NPU 驱动与应用运行支持方向

7.2 未来工作展望

【待完善】2-3 段，说明后续计划：

- 多核调度优化（参考 KRR 等工作）
- 虚拟内存优化（参考 CortenMM，取消 VMA 抽象）
- NPU 驱动完善与 AI 应用落地
- 组件进一步开源复用（已有 weak-map、extern-trait、scope-local 等发布计划）

参考文献

[1] TIAN H, OTHERS. Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB[Z]. 2025.

[2] BOOS K, LIYANAGE N, IJAZ R, et al. Theseus: An Experiment in Operating System Structure and State Management[C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020: 1-19.

[3] LEVY A, CAMPBELL B, GHENA B, et al. Multiprogramming a 64 kB Computer Safely and Efficiently[C]//Proceedings of the 26th Symposium on Operating Systems Principles (SOSP). 2017: 385-401.

[4] SCHUERMANN L, CAMPBELL B, GHENA B, et al. Tock: From Research to Securing 10 Million Computers[C]//Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP). 2025.

[5] GOOGLE. gVisor: An Application Kernel for Containers[Z]. 2026.

[6] TSAI C C, ARPACI-DUSSEAU A, ARPACI-DUSSEAU R. Graphene: A Library OS for Linux Multi-Process Applications[J]. USENIX ;login:, 2017, 42(2).

[7] SHEN Y, TIAN H, CHEN Y, et al. Occlum: Secure and Efficient Multitasking Inside a Single Enclave of Intel SGX[C]//Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2020: 1073-1087.

[8] ZHANG J, OTHERS. CortenMM: Efficient Memory Management with Strong Correctness Guarantees[C]//Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP). 2025.

[9] 朝倉水希. extern-trait: Opaque Types for Traits Using Link-Time Static Dispatch[Z]. 2025.

[10] ASTERINAS PROJECT. Asterinas: A Safe, Fast, and General OS Kernel[Z]. 2024.

[11] KLABNIK S, NICHOLS C. The Rust Programming Language[M]. No Starch Press, 2023.

[12] PANTER S, EISTY N. Rusty Linux: Advances in Rust for Linux Kernel Development[C]// Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). 2024.

[13] LI Z, OTHERS. Rust for Linux: Understanding the Security Impact of Rust in the Linux Kernel[C]//2024 Annual Computer Security Applications Conference (ACSAC). 2024.

[14] CUTLER C, KAASHOEK M F, MORRIS R T. The Benefits and Costs of Writing a POSIX Kernel in a High-Level Language[C]//13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). 2018.

[15] LANKES S, PICKARTZ S, BREITBART J. HermitCore: A Unikernel for Extreme Scale Computing[C]//Proceedings of the 6th International Workshop on Runtime and Operating Systems for Supercomputers. 2016.

[16] LANKES S, OTHERS. RustyHermit: A Scalable, Rust-Based Virtual Execution Environment[C]//International Conference on High Performance Computing. Cham: Springer International Publishing, 2020.

[17] SHARMA A, OTHERS. Rust for Embedded Systems: Current State and Open Problems[C]// Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024.

[18] LEVY A, OTHERS. Ownership is Theft: Experiences Building an Embedded OS in Rust[C]// Proceedings of the 8th Workshop on Programming Languages and Operating Systems. 2015.

[19] LEVY A, OTHERS. The Case for Writing a Kernel in Rust[C]//Proceedings of the 8th Asia-Pacific Workshop on Systems. 2017.

[20] LANKES S, BREITBART J, PICKARTZ S. Exploring Rust for Unikernel Development[C]// Proceedings of the 10th Workshop on Programming Languages and Operating Systems. 2019.

[21] KANTEE A. Flexible Operating System Internals: The Design and Implementation of the Anykernel and Rump Kernels[D]. 2012.

[22] PENG Y, OTHERS. Framekernel: A Safe and Efficient Kernel Architecture via Rust-based Intra-kernel Privilege Separation[C]//Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems. 2024.

[23] RUST LANG TEAM. RFC 3632: Externally Implementable Functions[Z]. 2024.

附录 A 外文资料的调研阅读报告（或书面翻译）

调研阅读报告题目（或书面翻译题目）

写出至少 5000 外文印刷字符的调研阅读报告或者书面翻译 1-2 篇（不少于 2 万外文印刷符）。

A.1 Lorem ipsum dolor.

A.1.1 Lorem ipsum dolor.

参考文献（或书面翻译对应的原文索引）

[1] 某某某. 信息技术与信息服务国际研讨会论文集: A 集［C］北京：中国社会科学出版社，1994.

附录 B 其他附录内容

【待完善】填写其他附录内容（如有）

致　谢

【待完善】 填写致谢内容

声　明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名：_____ 日 期：_____

综合论文训练记录表

<table>
<tr><td>学生姓名</td><td>王铮</td><td>学号</td><td>2022010862</td><td>班级</td><td>计 26</td></tr>
<tr><td>论文题目</td><td colspan="5">AIoT 操作系统性能优化关键技术研究</td></tr>
<tr><td>主要内容以及进度安排</td><td colspan="5">主要内容：

进度安排：

指导教师签字：

考核组组长签字：

年　月　日</td></tr>
<tr><td>中期考核意见</td><td colspan="5">考核组组长签字：

年　月　日</td></tr>
</table>

指导教师评语	指导教师签字： 年　月　日
评阅教师评语	评阅教师签字： 年　月　日
答辩小组评语	答辩小组组长签字： 年　月　日

总成绩：_____

教学负责人签字：_____

年　月　日